

Building Resilient Distributed Systems for StudySync

PDC Assignment Report

Mohid Faisal
BSCS23092

Context. StudySync follows the typical React + FastAPI pattern from Tech With Tim’s full-stack walk-through, then adds Clerk authentication/webhooks and an external LLM call. Once the system is exposed to concurrent users, unreliable delivery, and slow third-party dependencies, three distributed-systems failures appear: lost updates, webhook state divergence, and cascading latency when the LLM stalls.

Part 1: Analyze the Mess

1. Synchronization failure (lost update). The bug occurs in the read-modify-write cycle of the document API. Two clients read the same row at version v , edit locally, and then send writes that overwrite the whole document. In the naive flow, the database update statement does not verify whether the row is still at version v , so the second write succeeds even though the state changed after the first write. The conflict is not detected at the API boundary, and one user’s work disappears silently. This is a classic *lost update anomaly* caused by missing concurrency control.

2. Coordination failure (dropped webhook). Clerk sends subscription lifecycle events asynchronously. If the backend treats each webhook as a one-shot notification and updates premium state directly in the request handler, then any transient outage between Clerk and the backend can permanently desynchronize the systems. Once the cancellation event is dropped, StudySync has no durable record that it was expected, no retry-safe identifier to deduplicate replays, and no reconciliation job to repair the mismatch. The result is permanent state inconsistency: Clerk says “cancelled” while StudySync still marks the user as premium.

3. Fault-tolerance failure (blocking LLM dependency). The LLM call acts as a single point of failure because the API path waits on an external network dependency before it can complete the request. If that dependency hangs for 60 seconds, request workers remain occupied, the connection pool backs up, and unrelated users experience timeouts even though their work may not require the LLM. In distributed-systems terms, the service lacks isolation, timeouts, and failure containment, so one unhealthy downstream component amplifies latency across the whole application.

Part 2: Design a Better System

A. Sync fix: optimistic locking. Add a `version` column to each document. Every read returns the current version, and every update submits `expected_version`. The database performs an atomic conditional update: `UPDATE ... WHERE id = ? AND version = expected_version`. If one row is updated, the write succeeds and the version increments. If zero rows are updated, the API returns `409 Conflict` with the latest server copy so the client can refresh, merge, and retry. This preserves correctness without holding long-lived locks, which is ideal for collaborative editing workloads with moderate contention.

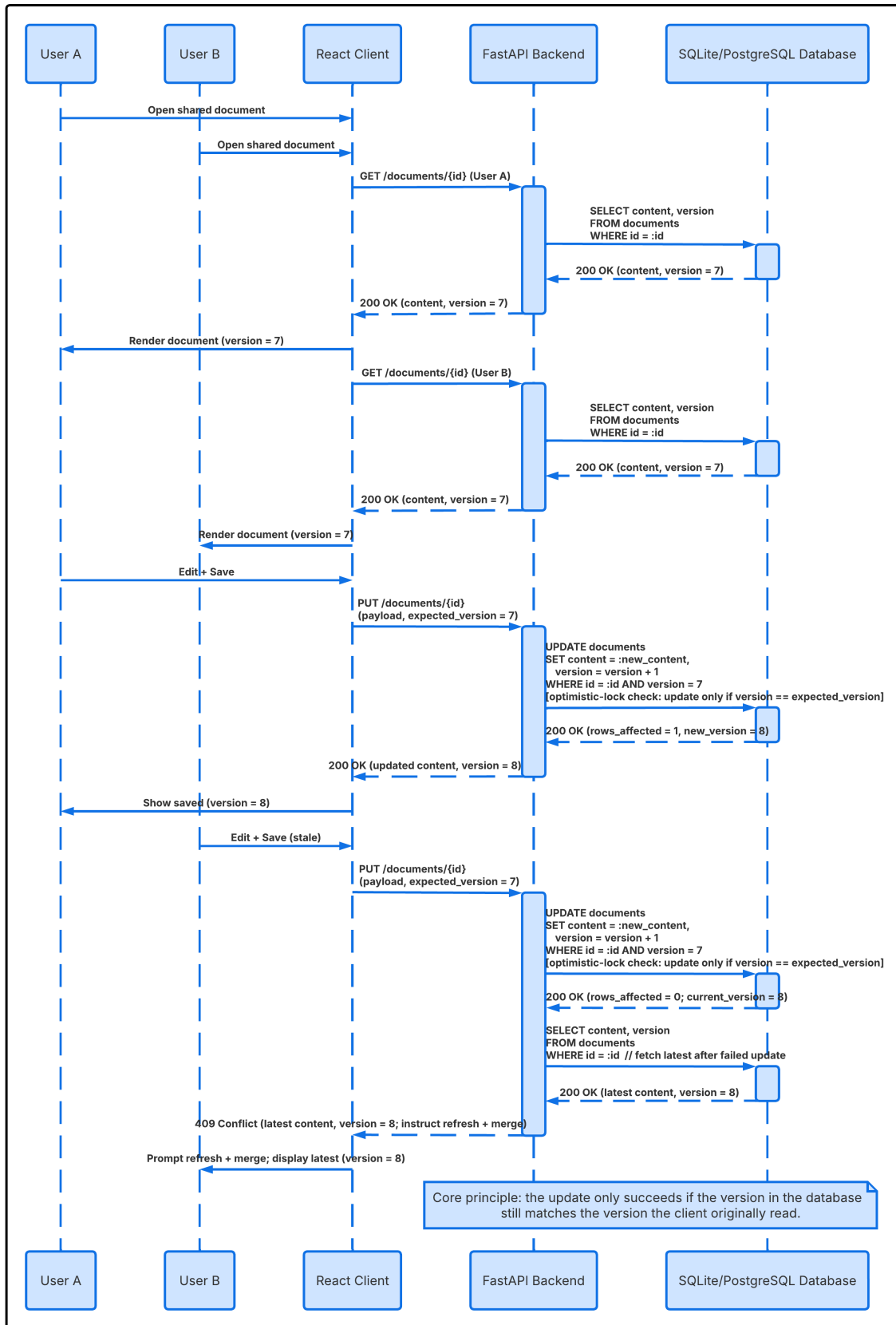


Figure 1: UML Sequence Diagram For Optimistic Locking Workflow

B. Coordination fix: durable webhook ingestion. The webhook handler should verify the Clerk signature, then persist the event to durable storage before applying business logic. Each event must carry an idempotency key such as Clerk’s event ID, stored in a processed-events table with a unique constraint. A background worker can then update subscription state and acknowledge duplicates safely. If downstream processing fails, the event stays in a retry queue with exponential backoff; after repeated failures it moves to a dead-letter queue for operator review. This design replaces “best effort” delivery with at-least-once processing plus deduplication and replay.

C. Fault-tolerance fix: circuit breaker with fallback. Wrap the LLM client in three protections: a short timeout, a circuit breaker, and a fallback response. Consecutive failures open the breaker, causing FastAPI to fail fast instead of waiting on a known-bad dependency. After a cool-down period, the breaker allows a probe request in half-open state; success closes it, and failure reopens it. During open state, the API should return a degraded but useful response, such as cached hints, a rule-based summary, or “AI assistance temporarily unavailable.” This trades perfect feature completeness for overall system responsiveness.

CAP and latency trade-offs. The optimistic-locking design chooses *consistency over blind availability*: conflicting writes are rejected instead of accepted and silently merged incorrectly. The webhook pipeline chooses *durability and eventual consistency* over immediate synchronous confirmation, because queued retries may delay downgrade visibility but prevent permanent divergence. The circuit breaker sacrifices some *feature availability* for lower tail latency and higher system availability: it is better to return a fast fallback than let one slow dependency exhaust the service. Across all three fixes, the architectural theme is to bound inconsistency, isolate failures, and make degraded behavior explicit instead of invisible.

Reference. Tech With Tim, *How To Build a FastAPI and React Full Stack App*. <https://www.youtube.com/watch?v=13tMEW8r6C0>